

IoT-Enabled Retail Analytics on Azure Databricks

creation of storage account:

The screenshot displays the Azure portal interface for a storage account named 'sagarcatalog'. The left sidebar shows the 'Storage accounts' section with a list of accounts: 'databricksmentor', 'nilaystore', 'sagarcatalog' (selected), and 'sagarstorage0'. The main content area shows the 'Overview' tab for 'sagarcatalog'. The 'Essentials' section lists key properties: Resource group (LabsKraft2027), Location (westus2), Subscription (LabsKraft), Subscription ID (d008c7cc-9795-4292-bf79-74ae52cda63d), Disk state (Available), and Tags. The 'Properties' section shows 'Data Lake Storage' settings: Hierarchical namespace (Enabled), Default access tier (Hot), and Blob anonymous access (Disabled). The 'Security' section shows 'Require secure transfer for REST API operations' (Enabled) and 'Storage account key access' (Enabled).

creation of access connector:

The screenshot displays the Azure portal interface for an access connector named 'sagarconnector'. The left sidebar shows the 'Access Connect... for Azure Databricks' section with a list of connectors: 'sagarconnector' (selected). The main content area shows the 'Overview' tab for 'sagarconnector'. The 'Essentials' section lists key properties: Resource group (LabsKraft2027), Location (West US 2), Subscription (LabsKraft), Subscription ID (d008c7cc-9795-4292-bf79-74ae52cda63d), and Tags. The 'State' is 'Succeeded' and the 'Resource ID' is '/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/...'. The 'Settings', 'Automation', and 'Help' sections are also visible.

giving access connector permission for storage:

The screenshot shows the Microsoft Azure portal interface. The main pane is titled 'Add role assignment' and shows the 'Storage Blob Data Contributor' role selected. The 'Assign access to' section has 'Managed identity' selected. The 'Members' section shows 'No members selected'. A sidebar titled 'Select managed identities' is open, showing a list of managed identities. The 'Access Connector for Azure Databricks (1)' identity is selected. The sidebar also shows a warning about ABAC conditions and a 'Selected members' section.

creation of uc:

The screenshot shows the Microsoft Azure Databricks interface. The main pane is titled 'Catalog Explorer' and shows the 'sagar_cat_project1' catalog. The 'About this catalog' section displays the following metadata:

Property	Value
Metastore Id	4f3b2018-fdc7-47a3-965c-db7d2dc65f0b
Created At	Aug 25, 2025, 09:13 AM
Created By	tredence14@michaelkloudkraft@hotmail.onmicrosoft.com
Updated At	Aug 25, 2025, 09:13 AM
Updated By	tredence14@michaelkloudkraft@hotmail.onmicrosoft.com
Storage Root	abfss://raw@sagarcatalog.dfs.core.windows.net/
Storage Location	abfss://raw@sagarcatalog.dfs.core.windows.net/_unitystorage/catalogs/00d8640c-c495-4e5e-b9c6-44707a58745

The 'Advanced' section shows 'Predictive Optimization' is enabled (Inherited).

creation of namespace and eventhub:

[Home](#) > [Event Hubs](#) >



Create Namespace ...

Event Hubs

Project Details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription *	LabsKraft
Resource group *	LabsKraft2027

[Create new](#)

Instance Details

Enter required settings for this namespace, including a price tier and configuring the number of units (capacity).

Namespace name *	sagarspace
Region *	West US 2
Pricing tier *	Basic
Throughput Units *	1

.servicebus.windows.net

i The region selected supports Availability zones. Your namespace will have Availability Zones enabled. [Learn more.](#)

[Browse the available plans and their features](#)

[Review + create](#)

[< Previous](#)

[Next: Advanced >](#)

portal.azure.com/#view/Microsoft_Azure_EventHub/CreateBlade/_provisioningContext~/%7B%22initialValues%22%3A%7B%22subscriptionId%22%3A%5B%22d008c7cc-9795-4292-bf79-74ae52c...

Microsoft Azure

Search resources, services, and docs (G+/)

Copilot

trredence14@michaelklo...
DEFAULT DIRECTORY (MICHAELKLO...

Home > Event Hubs >

Create Namespace ...

Event Hubs

Validating...

BasicsAdvancedNetworkingTagsReview + create

Event Hubs Namespace
by Microsoft

Basics

Namespace name	sagarhub
Subscription	LabsKraft
Resource group	LabsKraft2027
Region	West US 2
Pricing tier	Basic
Throughput Units	1
Availability Zones (Zone Redundancy)	Enabled

Networking

Connectivity method	Public access
---------------------	---------------

Security

Minimum TLS version	1.2
---------------------	-----

Create< PreviousNext >

24°C
Mostly cloudy

Search

Copilot

trredence14@michaelklo...
DEFAULT DIRECTORY (MICHAELKLO...

Home > Event Hubs > sagarspace >

Create Event Hub ...

Event Hubs

Validation succeeded.

BasicsCaptureReview + create

Event Hubs Instance
by Microsoft

Basics

Name	sagarhub
Partition count	1

Retention

Cleanup policy	Delete
Retention time (hrs)	6

Capture

Capture Status	Not Supported
----------------	---------------

Create< PreviousNext >

creating synapse workspace:

Microsoft Azure

Search resources, services, and docs (G+/)

Copilot

Home > Azure Synapse Analytics >

Create Synapse workspace

Validation succeeded

information with the provider(s) of the offering(s) for support, billing and other transactional activities. Microsoft does not provide rights for third-party offerings. For additional details see [Azure Marketplace Terms](#).

Basics

Subscription

LabsKraft

Resource group

LabsKraft2027

Region

West US 2

Workspace name

(new) sagars

Data Lake Storage Gen2 account

https://sagarstorage0.dfs.core.windows.net

Data Lake Storage Gen2 file system

raw

Managed resource group

None

Role assignments

The Storage Blob Data Contributor role will be assigned on the specified Data Lake Storage Gen2 account to both the workspace managed identity and the current user.

Security

Authentication method

Use both local and Microsoft Entra ID authentication

SQL Server admin login

sqladminuser

SQL Password

Double encryption

No

Create

< Previous

Next >

Download a template for automation

setting up synapse ->

linking my adls for storing old data and these can be pulled into azure dbx

Microsoft Azure

Synapse Analytics

sagars

Synapse live

Validate all

Publish all

Other users in your workspace may have access to modify this item. Do not use this item unless you trust all users who may have access to the workspace.

Workspace

Linked

Filter resources by name

Azure Data Lake Storage Gen2

sagars (Primary - sagarstorage0)

raw (Primary)

(Attached Containers)

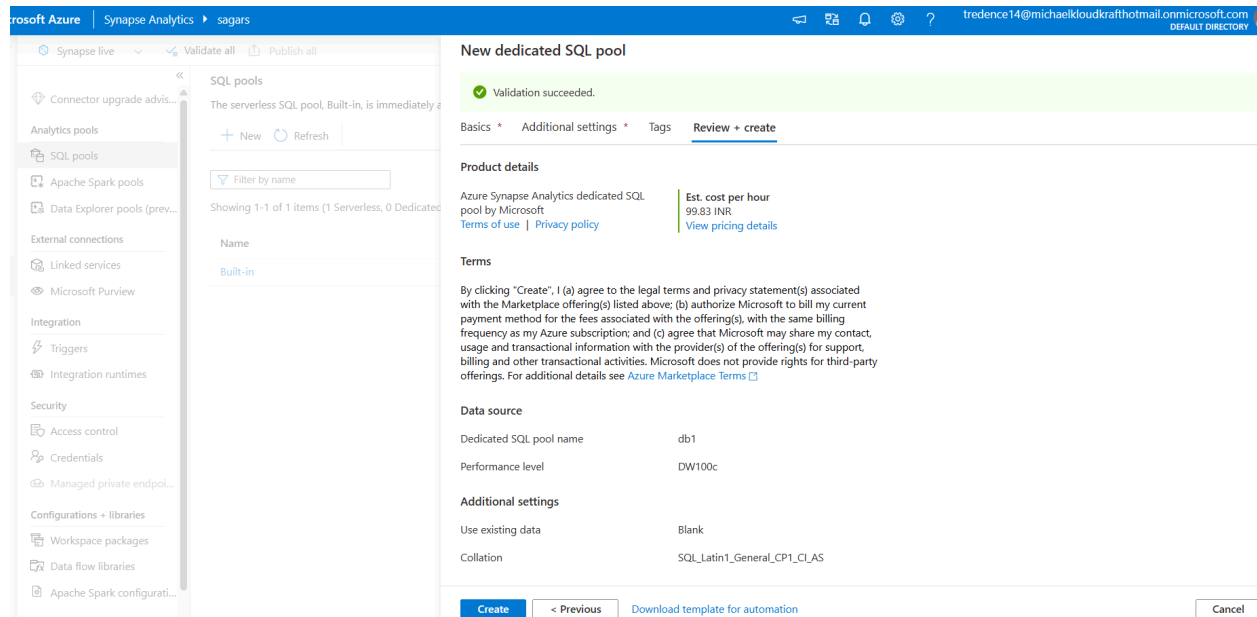
raw

retail_lot_data.csv

8/25/2025, 10:26:07 AM

130.8 KB

creation of new dedicated pool:



Microsoft Azure | Synapse Analytics | sagars

Synapse live | Validate all | Publish all

SQL pools

The serverless SQL pool, Built-in, is immediately available.

+ New Refresh

Filter by name

Showing 1-1 of 1 items (1 Serverless, 0 Dedicated)

Name

Built-in

New dedicated SQL pool

Validation succeeded.

Basics * Additional settings * Tags **Review + create**

Product details

Azure Synapse Analytics dedicated SQL pool by Microsoft

Terms of use | Privacy policy

Est. cost per hour: 99.83 INR

[View pricing details](#)

Terms

By clicking "Create", I (a) agree to the legal terms and privacy statement(s) associated with the Marketplace offering(s) listed above; (b) authorize Microsoft to bill my current payment method for the fees associated with the offering(s), with the same billing frequency as my Azure subscription; and (c) agree that Microsoft may share my contact, usage and transactional information with the provider(s) of the offering(s) for support, billing and other transactional activities. Microsoft does not provide rights for third-party offerings. For additional details see [Azure Marketplace Terms](#)

Data source

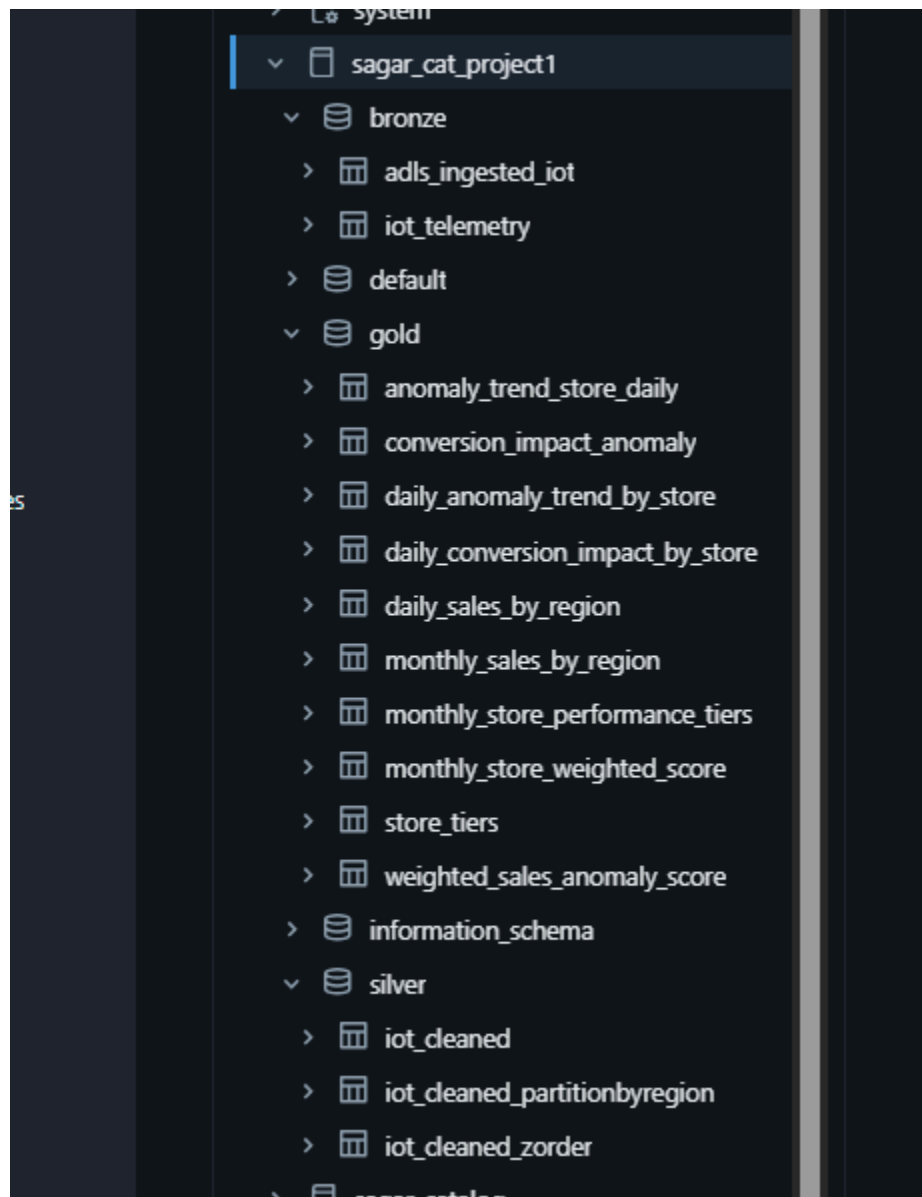
Dedicated SQL pool name	db1
Performance level	DW100c

Additional settings

Use existing data	Blank
Collation	SQL_Latin1_General_CP1_CLAS

Create < Previous Download template for automation Cancel

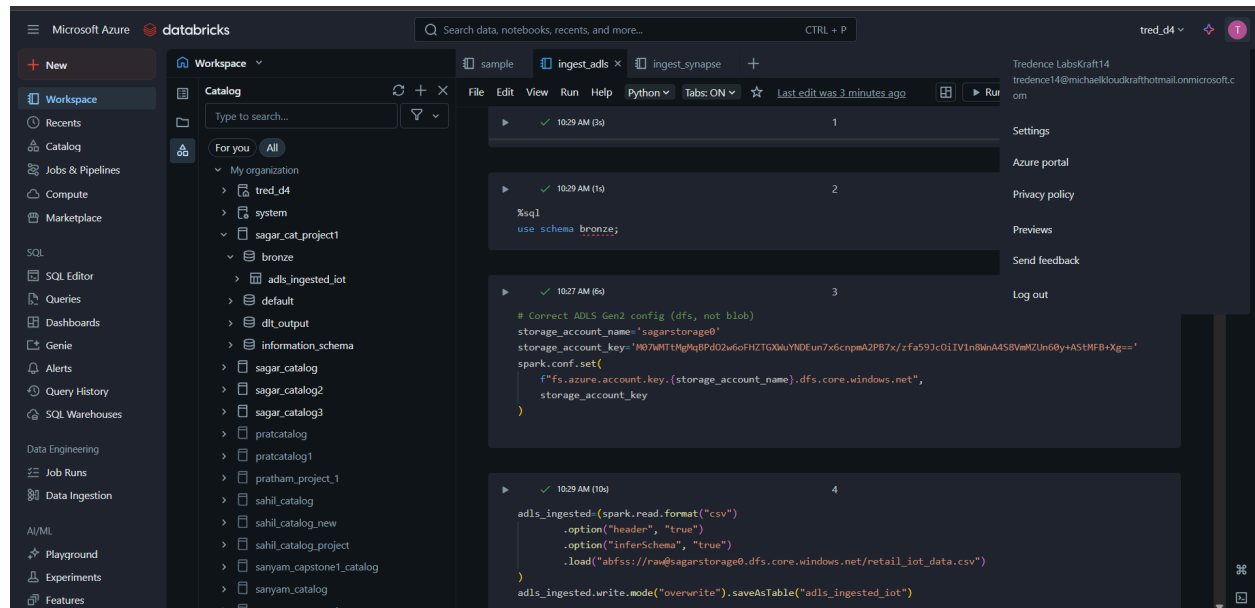
Quick Summary->



Ingestion mode->

i) ADLS

adls connection and storing file in sagar_cat_project1.bronze schema



adls_ingested_table->

OrderID	CustomerID	Region	StoreID	Product	Quantity	UnitPrice	SalesAmount	OrderTime	Device
db07ec0d-e8e7-408a-846e-c2d33048d4b	CUST-2824	East	Store-009	Mobile	2	322.1	644.2	2023-01-05T23:24:55.000+00:00	Temp_Sc
93d89258-6ef2-4bb8-9a71-cb590cb55f85	CUST-7912	East	Store-001	Laptop	2	503.69	1007.38	2023-01-30T05:24:09.000+00:00	RFID_Sc
f97636e9-3dab-48ad-aad4-0ca03d3111	CUST-9928	South	Store-008	Tablet	5	592.47	2962.35	2023-01-01T07:34:17.000+00:00	RFID_Sc
f20eae76-3e09-4380-8429-af1d6b76209a	CUST-3547	West	Store-011	Laptop	1	790.86	790.86	2023-01-18T10:14:28.000+00:00	POS_Ter
5dbcc9fd-0275-4427-a4b6-089b43465200	CUST-8527	East	Store-013	Laptop	11	621.7	6838.7	2023-01-18T13:20:07.000+00:00	Temp_Sc
c6bd3963-d239-42b7-b414-4e39289bc2da	CUST-5741	East	Store-008	Printer	1	791.25	791.25	2023-01-23T00:15:40.000+00:00	Temp_Sc
02fadf02-df5e-4527-94b1-9cb915a71209	CUST-6820	West	Store-009	Smartwatch	1	1237.81	1237.81	2023-01-09T07:23:27.000+00:00	Temp_Sc
90e7e190-9d09-47a3-a399-98562cf389dc	CUST-7216	North	Store-018	Mobile	3	1693.56	5080.68	2023-01-03T17:10:20.000+00:00	RFID_Sc
50f6c840-47fc-4117-47f6-78701a91199	CUST-7572	North	Store-003	Mobile	5	1758.92	8794.6	2023-01-16T06:37:34.000+00:00	RFID_Sc
77d4cd70-6019-4be0-bce4-51626ae52101	CUST-8517	West	Store-009	Mobile	2	1502.73	3005.46	2023-01-27T03:57:14.000+00:00	POS_Ter
6d65d67c-bc5e-43bf-9522-50e645af5f8f	CUST-7543	North	Store-008	Mobile	5	1012.36	5061.8	2023-01-03T06:53:40.000+00:00	RFID_Sc
d6c5b3f1-076e-4c24-a836-318d6588eedd	CUST-7916	East	Store-013	Tablet	5	1992.44	9962.2	2023-01-26T16:27:46.000+00:00	POS_Ter
18175f8f-6017-413a-9e20-c876857ee103	CUST-1188	East	Store-018	Printer	3	1548.77	4646.31	2023-01-17T12:19:59.000+00:00	RFID_Sc
54e45330-e78f-4d3d-9426-8eb38292bbd3	CUST-1053	North	Store-017	Printer	2	1039.98	2079.96	2023-01-06T03:58:39.000+00:00	Temp_Sc

ii)Event Hub:

we will connect to vm and then send data from local machine to eventhub which at the end will be recieved by DBX

creation of vm (namespace and eventhub already created) ->

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

fred@red14@michaelklo...
DEFAULT DIRECTORY (MICHAELKLO...)

Home > Compute infrastructure | Virtual machines >

Create a virtual machine

Help me choose the right VM size for my workloadHelp me create a VM optimized for high availabilityHelp me create a low cost VM

Validation passed

Preferred e-mail address

Preferred phone number

You have set SSH port(s) open to the internet. This is only recommended for testing. If you want to change this setting, go back to Basics tab.

Basics

Subscription

Resource group

Virtual machine name

Region

Availability options

Zone options

Availability zone

Security type

Enable secure boot

Enable vTPM

Integrity monitoring

LabsKraft

LabsKraft2027

sagarvm

Australia East

Availability zone

Self-selected zone

1

Trusted launch virtual machines

Yes

Yes

No

< Previous

Next >

Create

Download a template for automation

Give feedback

setting up vm session on local machine->

100%

sqladmin@sagarvm: ~ x + -

Microsoft Windows [Version 10.0.26100.4946]
(c) Microsoft Corporation. All rights reserved.

C:\Users\sagar>ssh sqladmin@20.5.40.164
The authenticity of host '20.5.40.164 (20.5.40.164)' can't be established.
ED25519 key fingerprint is SHA256:dMG/IXno/nV5rHkg3yW2cB8qD+bshQPvS+1UblZvo/U.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '20.5.40.164' (ED25519) to the list of known hosts.
sqladmin@20.5.40.164's password:
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.11.0-1018-azure x86_64)

* Documentation: <https://help.ubuntu.com>
* Management: <https://landscape.canonical.com>
* Support: <https://ubuntu.com/pro>

System information as of Fri Aug 22 04:23:48 UTC 2025

System load: 0.93 Processes: 27
Usage of /home: unknown Users logged in: 0
Memory usage: 5% IPv4 address for eth0: 10.10.10.2
Swap usage: 0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See <https://ubuntu.com/esm> or run: sudo pro status

Paste the below content:

import pandas as pd
import random

uploading my file to vm which is considered for ingestion

```
Microsoft Windows [Version 10.0.26100.4946]
(c) Microsoft Corporation. All rights reserved.

C:\Users\sagar>scp C:\Users\sagar\Downloads\iot_telemetry_data.csv sqladmin@20.5.40.164:/home/sqladmin/
sqladmin@20.5.40.164's password:
iot_telemetry_data.csv                                100% 1485      8.1KB/s   00:00

C:\Users\sagar>|
```

nano [send.py](#)

```
GNU nano 7.2 send.py
ls /home/sqladmin/ from azure.eventhub import EventHubProducerClient, EventData
import pandas as pd
import time
import json

# Load the CSV
df = pd.read_csv("iot_telemetry_data.csv")

# Event Hub connection
connection_str = "Endpoint=sb://sagarspace.servicebus.windows.net/;SharedAccessKeyName=RootManageSharedAccessKey;SharedAccessKey=2v97F8+tBbuM02W5qzsPlUAVWbYgfeZ+AEhBrynDE="
eventhub_name = "sagarhub"

producer = EventHubProducerClient.from_connection_string(conn_str=connection_str, eventhub_name=eventhub_name)

# Stream data row by row
for _, row in df.iterrows():
    data = row.to_dict()
    event_data_batch = producer.create_batch()
    event_data_batch.add(EventData(json.dumps(data)))
    producer.send_batch(event_data_batch)
    print(f'Sent: {data["OrderID"]}')
    time.sleep(1) # simulate 1 event/sec
```

sent file:

```

WARNING: The script faker is installed in '/home/sqladmin/.local/bin' which is not
Consider adding this directory to PATH or, if you prefer to suppress this warning,
Successfully installed faker-37.5.3 numpy-2.3.2 pandas-2.3.2 tzdata-2025.2
sqladmin@sagarvm:~$ python3 send.py
Sent: a1f3d6b7-45e8-4a30-8e9b-1f9d2caa6710
Sent: b7d4f3c2-9c23-4a0a-a4b5-f0317b53f8f5
Sent: c6e8f5a1-58a9-4980-a71c-34266c5db412
Sent: d8f4c9b2-92d2-4a9f-a6b0-15db9d214e62
Sent: e3c9a2f1-73a4-41e8-b4fd-62b0a930ed7b
Sent: f5d6b8c3-84b9-4a77-a9c1-9d1e1cbf6d2e
Sent: g7a2d1e4-91c5-4f21-bf28-7bc0e55d9183
Sent: h9f8e3d5-11a2-48a4-9c2b-64b0b11c6d45
Sent: i2d4b7a6-5c31-4c29-bd38-27f7a04d0c9e

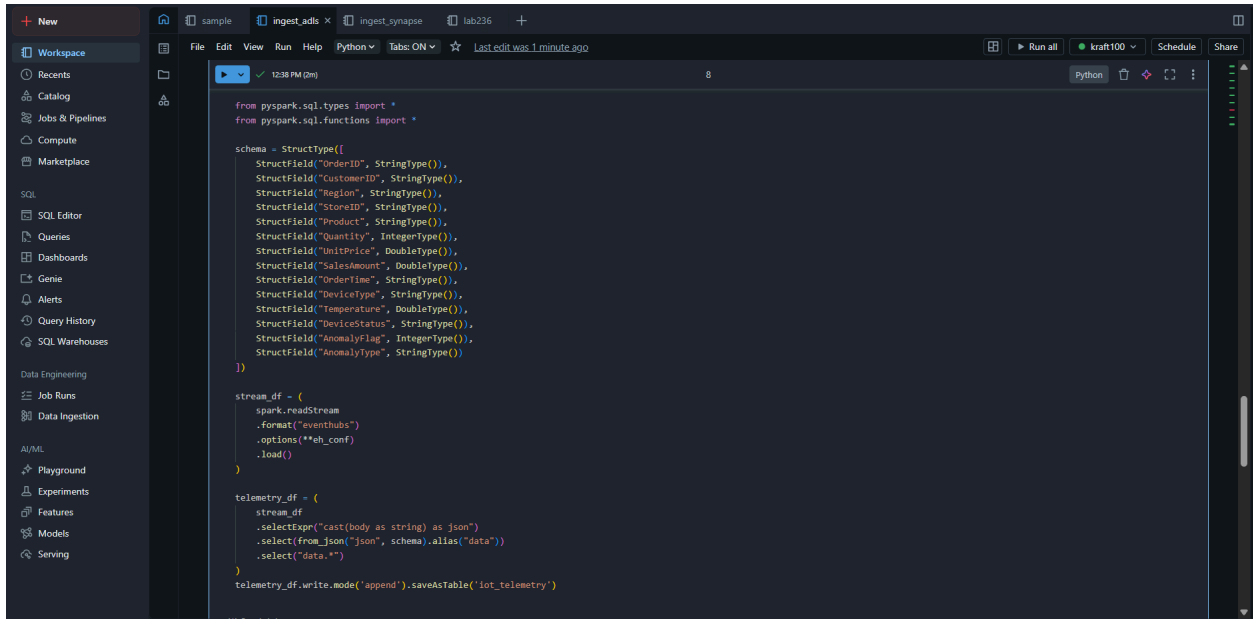
```

File received on EH

The screenshot shows the Azure Data Explorer interface for the 'sagarhub' Event Hubs instance. The left sidebar contains navigation options: Overview, Access control (IAM), Diagnose and solve problems, Data Explorer (selected), Resource visualizer, Settings, Entities, Features, Automation, and Help. The main pane displays a table of 10 received events. The table has columns for Sequence Number, Offset, Partition ID, Enqueued Time, Content Type, Message ID, and Event Body. The Event Body column shows JSON-like strings with 'OrderID' values corresponding to the UUIDs sent in the terminal above. Below the table, there are tabs for 'Event body' and 'Event properties', and a 'View events' button at the bottom.

Sequence Nu...	Offset	Partition ID	Enqueued Time	Content Type	Message ID	Event Body
0	0	0	Mon, Aug 25, 25, 12:05:40 PM G...			{"OrderID": "a1f3d6b7-45e8-4a30-8e9b-1f9d2caa6710"}
1	424	0	Mon, Aug 25, 25, 12:05:42 PM G...			{"OrderID": "b7d4f3c2-9c23-4a0a-a4b5-f0317b53f8f5"}
2	840	0	Mon, Aug 25, 25, 12:05:43 PM G...			{"OrderID": "c6e8f5a1-58a9-4980-a71c-34266c5db412"}
3	1264	0	Mon, Aug 25, 25, 12:05:44 PM G...			{"OrderID": "d8f4c9b2-92d2-4a9f-a6b0-15db9d214e62"}
4	1688	0	Mon, Aug 25, 25, 12:05:45 PM G...			{"OrderID": "e3c9a2f1-73a4-41e8-b4fd-62b0a930ed7b"}
5	2112	0	Mon, Aug 25, 25, 12:05:46 PM G...			{"OrderID": "f5d6b8c3-84b9-4a77-a9c1-9d1e1cbf6d2e"}
6	2552	0	Mon, Aug 25, 25, 12:05:48 PM G...			{"OrderID": "g7a2d1e4-91c5-4f21-bf28-7bc0e55d9183"}
7	2968	0	Mon, Aug 25, 25, 12:05:49 PM G...			{"OrderID": "h9f8e3d5-11a2-48a4-9c2b-64b0b11c6d45"}
8	3384	0	Mon, Aug 25, 25, 12:05:50 PM G...			{"OrderID": "i2d4b7a6-5c31-4c29-bd38-27f7a04d0c9e"}
9	3800	0	Mon, Aug 25, 25, 12:05:51 PM G...			{"OrderID": "j3e9c5b8-7a44-4b29-bd38-27f7a04d0c9e"}

Reading data from eventhub and storing into my bronze schema as `iot_telemetry`



```
from pyspark.sql.types import *
from pyspark.sql.functions import *

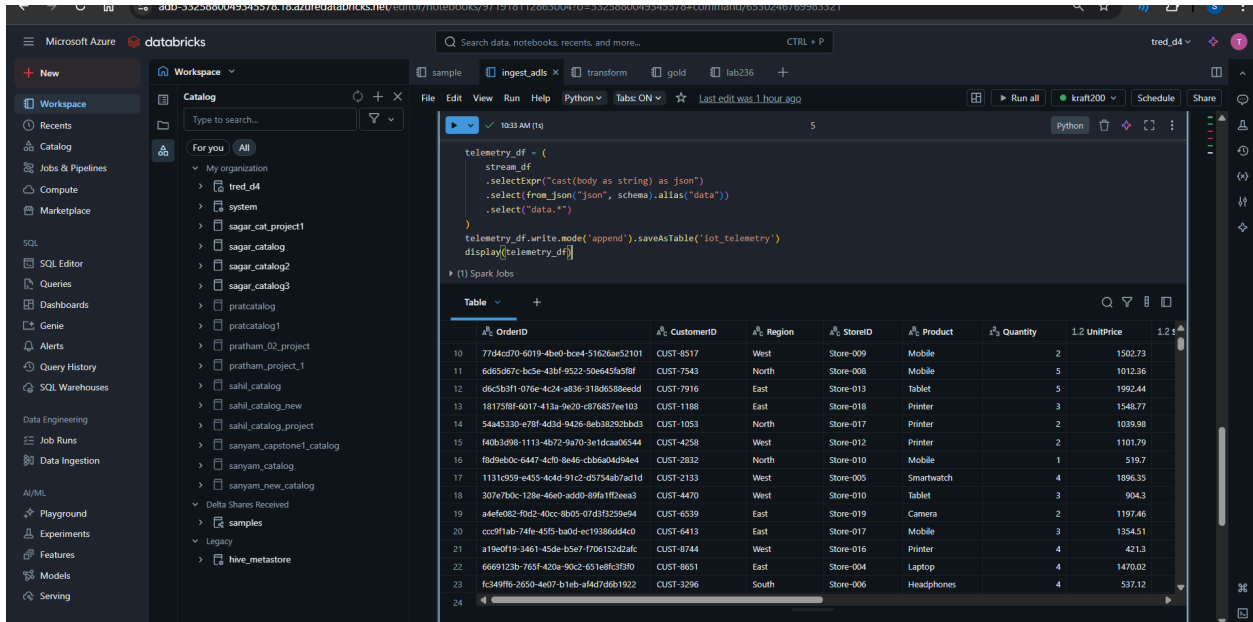
schema = StructType([
    StructField("OrderID", StringType()),
    StructField("CustomerID", StringType()),
    StructField("Region", StringType()),
    StructField("StoreID", StringType()),
    StructField("Product", StringType()),
    StructField("Quantity", IntegerType()),
    StructField("UnitPrice", DoubleType()),
    StructField("SalesAmount", DoubleType()),
    StructField("OrderTime", StringType()),
    StructField("DeviceType", StringType()),
    StructField("Temperature", DoubleType()),
    StructField("DeviceStatus", StringType()),
    StructField("AnomalyFlag", IntegerType()),
    StructField("AnomalyType", StringType())
])

stream_df = (
    spark.readStream
    .format("eventhubs")
    .options(**eh_conf)
    .load()
)

telemetry_df = (
    stream_df
    .selectExpr("cast(body as string) as json")
    .select(from_json("json", schema).alias("data"))
    .select("data.*")
)

telemetry_df.write.mode('append').saveAsTable('lot_telemetry')
```

Data streamed in from event_hub->



```
telemetry_df = (
    stream_df
    .selectExpr("cast(body as string) as json")
    .select(from_json("json", schema).alias("data"))
    .select("data.*")
)

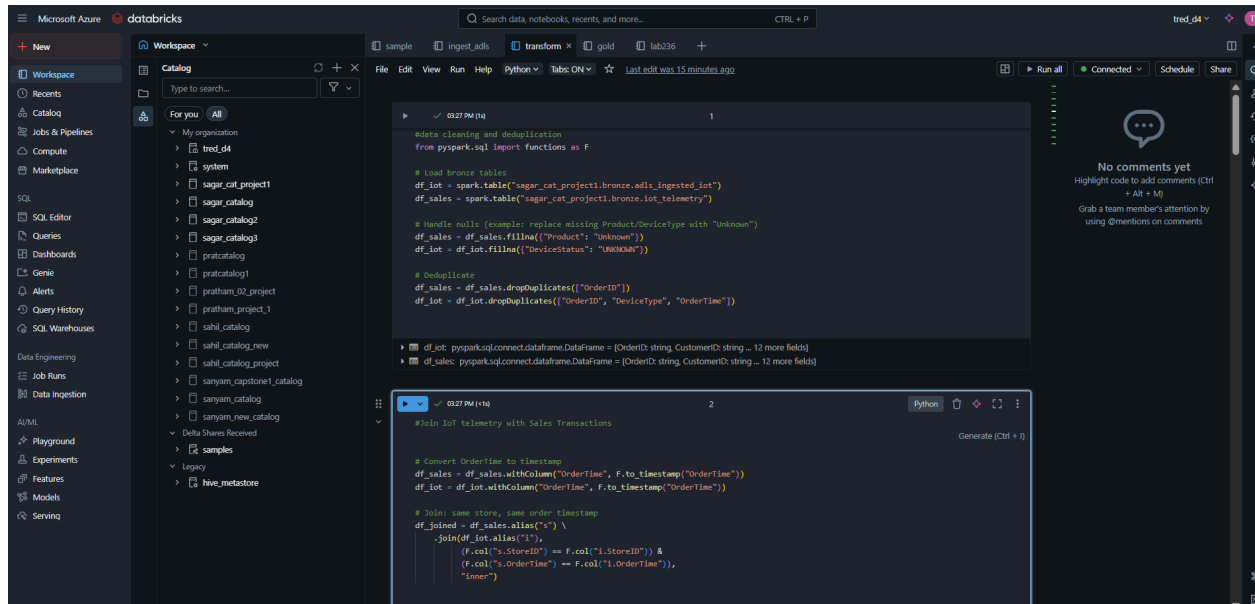
telemetry_df.write.mode('append').saveAsTable('lot_telemetry')
display(telemetry_df)
```

	OrderID	CustomerID	Region	StoreID	Product	Quantity	UnitPrice
10	77d4cd70-6019-4be0-bce4-3102bae32101	CUST-8517	West	Store-009	Mobile	2	1502.73
11	6dd5d67c-bc5e-43bf-9522-50e645a5f8f	CUST-7543	North	Store-008	Mobile	5	1012.36
12	d6c2b3f1-076e-4c24-a836-318d6588eedd	CUST-7916	East	Store-013	Tablet	5	1992.44
13	18175f8f-6017-413a-9e20-c876657ee103	CUST-1188	East	Store-018	Printer	3	1548.77
14	54a45330-e78f-4d3d-9426-8eb38292bbd3	CUST-1053	North	Store-017	Printer	2	1039.98
15	f4083d98-1113-4b72-9a70-3e1dca906544	CUST-4258	West	Store-012	Printer	2	1101.79
16	8d98eb0c-6447-4cf0-8e4e-cbb6a04d94e4	CUST-2832	North	Store-010	Mobile	1	519.7
17	1131c959-e455-4c4d-91c2-d5754ab7ad1d	CUST-2133	West	Store-005	Smartwatch	4	1896.35
18	307e7b0c-128e-46e0-ad00-89fa1f2eaa3	CUST-4470	West	Store-010	Tablet	3	904.3
19	a4efe082-f0d2-40cc-8b05-07d3f3259e94	CUST-6339	East	Store-019	Camera	2	1197.46
20	ccc9f1ab-74fe-43f5-ba0d-ect9386d44c0	CUST-6413	East	Store-017	Mobile	3	1354.51
21	a19e0f19-3461-45de-b5e7-7f06152d2afc	CUST-8744	West	Store-016	Printer	4	421.3
22	6669123b-765f-420a-90c2-631e8fc3f9f0	CUST-8651	East	Store-004	Laptop	4	1470.02
23	fc349f6-2650-4e07-b1eb-a447d0b1922	CUST-3296	South	Store-006	Headphones	4	537.12

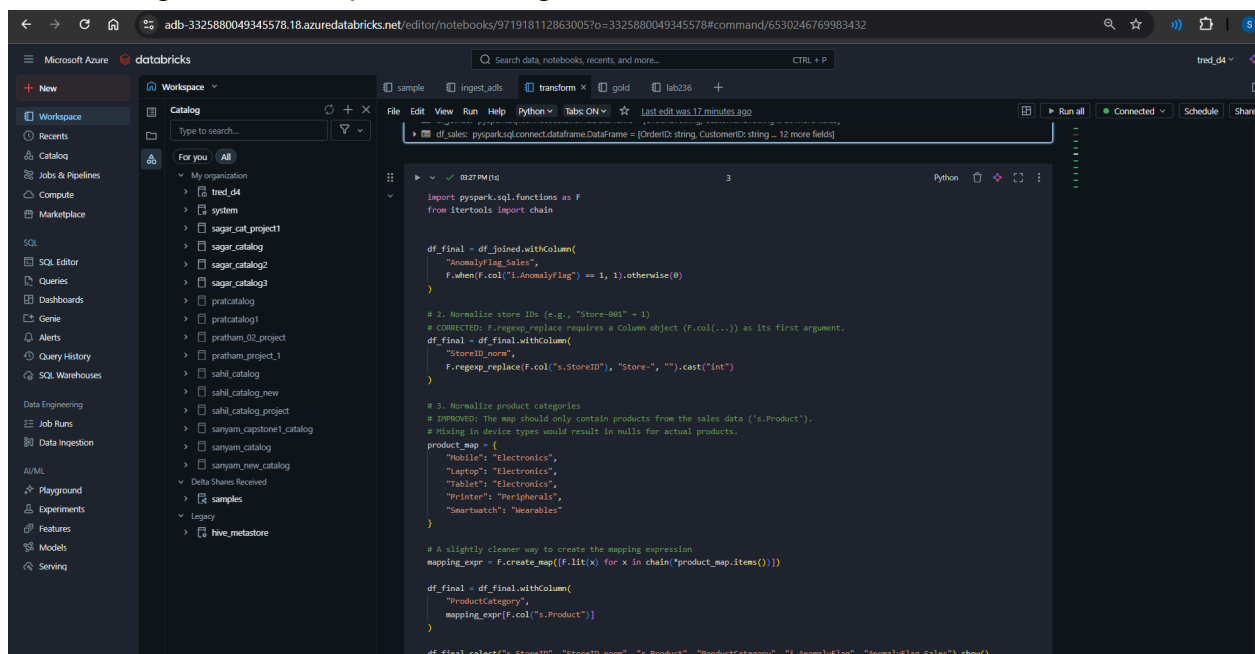
Both table from adls and eventhub stored in my bronze schema of catalog.

Transformation:

Removal of data quality issues and renaming column



Normalizing storeid and product categories



The screenshot shows the Databricks workspace interface. On the left is a sidebar with navigation options like Workspace, Catalog, Recent, and Jobs & Pipelines. The main area displays a Python notebook with the following code:

```
# A slightly cleaner way to create the mapping expression
mapping_expr = F.create_map([F.lit(x) for x in chain("product_map.items()")])

df_final = df_final.withColumn(
    "ProductCategory",
    mapping_expr[F.col("s.Product")]
)

df_final.select("s.StoreID", "StoreID_name", "s.Product", "ProductCategory", "l.AnomalyFlag", "AnomalyFlag_Sales").show()
```

Below the code, a table of results is displayed:

StoreID	StoreID_name	Product	ProductCategory	AnomalyFlag	AnomalyFlag_Sales
[Store-003]	3	Headphones	NULL	0	0
[Store-020]	20	Camera	NULL	1	1
[Store-007]	7	Laptop	Electronics	0	0
[Store-019]	19	Tablet	Electronics	0	0
[Store-001]	1	Smartwatch	Wearables	0	0
[Store-004]	4	Headphones	NULL	0	0
[Store-006]	6	Headphones	NULL	1	1
[Store-002]	2	Smartwatch	Wearables	0	0
[Store-010]	10	Printer	Peripherals	0	0
[Store-005]	5	Headphones	NULL	0	0
[Store-003]	3	Laptop	Electronics	0	0
[Store-020]	20	Printer	Peripherals	0	0
[Store-015]	15	Smartwatch	Wearables	0	0
[Store-018]	18	Tablet	Electronics	0	0
[Store-003]	3	Smartwatch	Wearables	0	0
[Store-015]	15	Tablet	Electronics	1	1
[Store-010]	10	Headphones	NULL	1	1
[Store-004]	4	Headphones	NULL	0	0

Finally saving in silver schema of my catalog

Partitioning by region and zordering->

```
from pyspark.sql import functions as F

# 1. Configuration
CATALOG = "sagar_cat_project1"
SCHEMA = "silver"
TABLE_NAME = "iot_cleaned"
FULL_TABLE_NAME = f"{CATALOG}.{SCHEMA}.{TABLE_NAME}"

# Load the existing Silver table
df = spark.table(FULL_TABLE_NAME)

(df.write
 .format("delta")
 .mode("overwrite")
 .option("overwriteSchema", "true")
 .partitionBy("Region") # Apply partitioning by Region
 .saveAsTable(f"sagar_cat_project1.silver.iot_cleaned_partitionbyregion"))

print("Table successfully rewritten and partitioned by Region.")

# 3. (Recommended) Verify the partitioning
print("\nVerifying table properties...")
display(spark.sql(f"DESCRIBE DETAIL {FULL_TABLE_NAME}"))
```

> See performance (3)

df: pyspark.sql.connect.dataframe.DataFrame = [OrderID: string, CustomerID: string ... 12 more fields]

Table successfully rewritten and partitioned by Region.

Verifying table properties...

Table	format	id	name	description	location
1	delta	29904b60-4ade-465d-a47e-e03496ec33...	sagar_cat_project1.silver.iot_clean...	null	> abfss://raw8@sagarcatalog.dfs.core.windows

Have a look on all the tables that is created in silver region->

iot_cleaned->

display(spark.table(full_table_name).limit(10))

> See performance (3)

--- Displaying top 5 rows of: sagar_cat_project1.silver.iot_cleaned ---

OrderID	CustomerID	StoreID	ProductCategory	Region	Quantity	UnitPrice	SalesAmount	OrderTime	DeviceType	Temperature	DeviceStatus	AnomalyFlag	Sales	AnomalyType
655827a9-9a4f-4301-948f-3c5702bdc4	CUST-9476	3	Electronics	East	3	833.22	2499.66	2023-01-08T23:27:07.000+00...	POS_Terminal	18.43	OK	0		
91842669-9f1a-4699-801a-cd3306c8ba0	CUST-21373	20	Electronics	North	9	552.72	4924.48	2023-01-01T06:22:53.000+00...	Temp_Sensor	23.21	MISMATCH	1		RFD_Mismatch
013d4803-f1a9-4899-aed5-11917b31717...	CUST-2860	7	Electronics	West	2	1604.24	3208.48	2023-01-16T01:48:21.000+00...	POS_Terminal	21.4	OK	0		
7823386a-584c-4d3a-b893-1071c13d6a...	CUST-2844	19	Electronics	North	4	953.26	3813.04	2023-01-02T13:31:13.000+00...	Temp_Sensor	16.64	OK	0		
28c37d95-2556-42a9-b855-cd7c9988ba	CUST-4842	1	Wearables	East	1	1818.42	1818.42	2023-01-26T20:56:11.000+00...	Temp_Sensor	21.46	OK	0		
18c4d4-15b0-4caf-aed5-3d21c9c8d74	CUST-2843	4	Wearables	West	1	447.28	447.28	2023-01-14T19:01:10.000+00...	RFD_Scanner	23.94	OK	0		
044c24e-5477-4d9e-b847-5b32c9c9d92	CUST-2609	6	Wearables	South	10	758.98	7589.8	2023-01-15T18:36:04.000+00...	RFD_Scanner	18.68	MISMATCH	1		RFD_Mismatch
77744d56-bba0-4c09-b04a-da067896a84	CUST-2609	7	Wearables	West	1	945.77	945.77	2023-01-26T08:34:33.000+00...	RFD_Scanner	18.03	OK	0		
b772d890-846a-46a3-479c-478a64101b	CUST-9666	19	Peripherals	North	3	1125.51	3376.53	2023-01-30T11:45:01.000+00...	RFD_Scanner	24.28	OK	0		
bc9e732-438f-4153-b894-788557a33c...	CUST-1472	6	Wearables	West	5	76.67	383.35	2023-01-20T19:20:36.000+00...	RFD_Scanner	19.57	OK	0		

10 rows | 2.25s runtime

--- Displaying top 5 rows of: sagar_cat_project1.silver.iot_cleaned_partitionbyregion ---

after partitioning->

display(spark.table(full_table_name).limit(10))

> See performance (3)

--- Displaying top 5 rows of: sagar_cat_project1.silver.iot_cleaned_partitionbyregion ---

OrderID	CustomerID	StoreID	ProductCategory	Region	Quantity	UnitPrice	SalesAmount	OrderTime	DeviceType	Temperature	DeviceStatus	AnomalyFlag	Sales	AnomalyType
01b6b800-41aa-4969-aed5-1317b31717...	CUST-2860	7	Electronics	West	2	1604.24	3208.48	2023-01-16T01:48:21.000+00...	POS_Terminal	21.4	OK	0		
18c4d4-15b0-4caf-aed5-3d21c9c8d74	CUST-2843	4	Wearables	West	1	447.28	447.28	2023-01-14T19:01:10.000+00...	RFD_Scanner	23.94	OK	0		
77744d56-bba0-4c09-b04a-da067896a84	CUST-2609	7	Wearables	West	1	945.77	945.77	2023-01-26T08:34:33.000+00...	RFD_Scanner	18.03	OK	0		
bc9e732-438f-4153-b894-788557a33c...	CUST-1472	6	Wearables	West	5	76.67	383.35	2023-01-20T19:20:36.000+00...	RFD_Scanner	19.57	OK	0		
a1b4d403-14c3-43ce-ab09-baef90c0d140	CUST-5626	18	Electronics	West	1	309.26	309.26	2023-01-29T04:11:45.000+00...	RFD_Scanner	23.95	OK	0		
eb595989-b152-418f-9666-a51b777e61	CUST-4259	10	Electronics	West	10	1362.73	13627.3	2023-01-24T15:58:16.000+00...	Temp_Sensor	21.13	MISMATCH	1		RFD_Mismatch
2be1eaa-d822-4c83-df5d-dbe11940020	CUST-4610	11	Electronics	West	3	1057.18	3171.54	2023-01-01T11:19:42.000+00...	Temp_Sensor	23.83	OK	0		
3076790c-123e-46e3-aed5-8991f2cea3	CUST-4470	10	Electronics	West	3	904.3	2712.9	2023-01-26T03:01:10.000+00...	POS_Terminal	18.85	OK	0		
451212d3-8a47-49a4-aed5-37b6b76272...	CUST-5056	10	Electronics	West	1	1825.26	1825.26	2023-01-04T04:18:47.000+00...	POS_Terminal	18.6	OK	0		
ac7f5a2-17ba-4845-803a-7d3a2f532	CUST-2846	4	Electronics	West	3	1600.05	4800.15	2023-01-17T11:03:46.000+00...	Temp_Sensor	24.57	OK	0		

10 rows | 2.25s runtime

partitioning zorder->

10 rows | 2.25s runtime

--- Displaying top 5 rows of: sagar_cat_project1.silver.iot_cleaned_order ---

	OrderID	CustomerID	StoreID	ProductCategory	Region	Quantity	UnitPrice	SalesAmount	OrderTime	DeviceType	Temperature	DeviceStatus	AnomalyFlag_Sales	AnomalyType
1	323c3d1e-15ec-4ff5-bc94-18ed7700ed	CUST-5643	2	Electronics	West	3	652.34	1957.02	2023-01-06T23:50:45.000+00...	POS_Terminal	24.98	OK	0	
2	adb193e5-8676-49a3-bbda-097ba3c089...	CUST-4066	6	Electronics	West	5	555.67	2778.35	2023-01-06T03:53:03.000+00...	POS_Terminal	35.33	OVERHEAT	1	Overheat
3	93205e99-7924-415c-bc50-0274d78c70a3	CUST-3960	1	Electronics	West	4	1611	6444	2023-01-06T23:21:29.000+00...	Rfid_Scanner	18.9	OK	0	
4	4210a853-dae6-4cd9-915c-4560837981...	CUST-3598	17	Electronics	West	1	708.58	708.58	2023-01-06T12:47:01.000+00...	Temp_Sensor	19.59	OK	0	
5	b7cdaeb0-ad5b-4a05-ad8a-958d734a20...	CUST-5748	19	Wearables	West	13	710.43	9235.59	2023-01-06T05:12:5.000+00...	Temp_Sensor	21.35	MISMATCH	1	Rfid_Mismatch
6	e761c1c16-5ec1-484f-9707-954477d3a54	CUST-2776	18	Peripherals	West	4	475.85	1903.4	2023-01-06T03:47:23.000+00...	POS_Terminal	23.01	OK	0	
7	ef39a9a6-a16d-4779-b3b1-c9d7ceec83...	CUST-5291	15	Peripherals	West	4	1143.77	4579.08	2023-01-06T19:02:03.000+00...	Rfid_Scanner	24.62	OK	0	
8	718a4d55-780e-4a43-d184-bc35970c3a4d	CUST-5673	6	Peripherals	West	2	364.45	728.9	2023-01-06T16:13:46.000+00...	Rfid_Scanner	24.96	OK	0	
9	2a32a6d3-7188-4ba2-d8a3-0818d3454db	CUST-1101	20	Wearables	West	1	1430.75	1430.75	2023-01-06T19:47:00.000+00...	Rfid_Scanner	24	OK	0	
10	ed56f6d7-c399-4876-836e-c1d8d5c6d9e	CUST-8405	3	Electronics	North	4	220.25	881	2023-01-19T23:10:41.000+00...	Rfid_Scanner	20.02	OK	0	

10 rows | 2.25s runtime

Aggregation & Enrichment Layer (Gold)->

```
from pyspark.sql import functions as F
from pyspark.sql.window import Window

CATALOG = "sagar_cat_project1"
SILVER_SCHEMA = f"{CATALOG}.silver"
GOLD_SCHEMA = f"{CATALOG}.gold"
BASE_TABLE = f"{SILVER_SCHEMA}.iot_cleaned"

# Create the Gold schema if it doesn't exist
spark.sql(f"CREATE SCHEMA IF NOT EXISTS {GOLD_SCHEMA}")

# Load the base Silver table
base_df = spark.table(BASE_TABLE)

# Standardize column names to ensure consistency
# (Handles cases like 'sales_amount' vs 'SalesAmount' and 'StoreID_norm' vs 'StoreID')
if "sales_amount" in base_df.columns and "SalesAmount" not in base_df.columns:
    base_df = base_df.withColumnRenamed("sales_amount", "SalesAmount")
if "StoreID_norm" in base_df.columns and "StoreID" not in base_df.columns:
    base_df = base_df.withColumnRenamed("StoreID_norm", "StoreID")

# Ensure correct data types for calculations
base_df = (base_df
            .withColumn("OrderTime", F.to_timestamp("OrderTime")))
```



```

        .withColumn("SalesAmount", F.col("SalesAmount").cast("double"))
        .withColumn("Quantity", F.col("Quantity").cast("int"))
    )

# Derive common date and month grains for aggregation
base_df = (base_df
    .withColumn("order_date", F.to_date("OrderTime"))
    .withColumn("order_month", F.date_trunc("month", "OrderTime"))
)

# -----
# 1) KPI: Daily and Monthly Sales per Region
# -----
print("Creating sales KPIs per region...")

# Daily aggregation
daily_sales = (base_df
    .groupBy("order_date", "Region")
    .agg(
        F.sum("SalesAmount").alias("total_sales"),
        F.sum("Quantity").alias("total_units_sold"),
        F.countDistinct("OrderID").alias("distinct_orders")
    )
)

(daily_sales.write.format("delta").mode("overwrite")
    .saveAsTable(f"{GOLD_SCHEMA}.daily_sales_by_region"))

# Monthly aggregation
monthly_sales = (base_df
    .groupBy("order_month", "Region")
    .agg(
        F.sum("SalesAmount").alias("total_sales"),
        F.sum("Quantity").alias("total_units_sold"),
        F.countDistinct("OrderID").alias("distinct_orders")
    )
)

(monthly_sales.write.format("delta").mode("overwrite")
    .saveAsTable(f"{GOLD_SCHEMA}.monthly_sales_by_region"))

```

```

# -----
# 2) KPI: Device Anomaly Trend per Store
# -----
print("Creating device anomaly trend KPI...")

anomaly_trend = (base_df
    .groupBy("order_date", "StoreID")
    .agg(
        F.sum("AnomalyFlag_Sales").alias("anomaly_count"),
        F.count("OrderID").alias("total_transactions")
    )
    .withColumn("anomaly_rate", F.col("anomaly_count") /
F.col("total_transactions"))
)

(anomaly_trend.write.format("delta").mode("overwrite")
    .saveAsTable(f"{GOLD_SCHEMA}.daily_anomaly_trend_by_store"))

# -----
# 3) KPI: Conversion Rate Impact from Device Failures
# -----
print("Creating conversion rate impact KPI...")

# We define "conversion" as the ratio of successful orders to total orders at a
store-day level
daily_impact = (base_df
    .groupBy("order_date", "StoreID")
    .agg(
        F.sum(F.when(F.col("AnomalyFlag_Sales") == 1,
1).otherwise(0)).alias("anomaly_orders"),
        F.count("OrderID").alias("total_orders")
    )
    .withColumn("successful_orders", F.col("total_orders") -
F.col("anomaly_orders"))
    .withColumn("success_rate", F.col("successful_orders") /
F.col("total_orders"))
    .withColumn("had_anomaly_day", F.when(F.col("anomaly_orders") > 0,
1).otherwise(0))

```

```

)

(daily_impact.write.format("delta").mode("overwrite")
  .saveAsTable(f"{GOLD_SCHEMA}.daily_conversion_impact_by_store"))

# -----
# 4) Enrichment: Tag Stores into Performance Tiers
# -----

print("Enriching stores with performance tiers...")

# Calculate total sales per store for each month
store_monthly_sales = base_df.groupBy("order_month",
  "StoreID").agg(F.sum("SalesAmount").alias("total_sales"))

# Use a window function to rank stores within each month
w_spec = Window.partitionBy("order_month").orderBy(F.col("total_sales").desc())
store_ranks = store_monthly_sales.withColumn("rank",
  F.percent_rank().over(w_spec))

# Assign tiers based on percentile rank
store_tiers = store_ranks.withColumn("performance_tier",
  F.when(F.col("rank") <= 0.2, "High")
    .when((F.col("rank") > 0.2) & (F.col("rank") <= 0.8), "Medium")
    .otherwise("Low"))

)

(store_tiers.select("order_month", "StoreID", "total_sales",
  "performance_tier")
  .write.format("delta").mode("overwrite")
  .saveAsTable(f"{GOLD_SCHEMA}.monthly_store_performance_tiers"))

# -----
# 5) Enrichment: Compute Weighted Sales vs. Anomalies Score
# -----

print("Enriching stores with a weighted performance score...")

# Get monthly sales and anomaly counts per store
monthly_kpis = base_df.groupBy("order_month", "StoreID").agg(
  F.sum("SalesAmount").alias("sales"),
  F.sum("AnomalyFlag_Sales").alias("anomalies")
)

```

```

)

# Define a window to normalize scores across all stores within a month
w_monthly = Window.partitionBy("order_month")

# Calculate min-max normalized scores for sales and anomalies
normalized_kpis = (monthly_kpis
    .withColumn("max_sales", F.max("sales").over(w_monthly))
    .withColumn("min_sales", F.min("sales").over(w_monthly))
    .withColumn("max_anomalies", F.max("anomalies").over(w_monthly))
    .withColumn("min_anomalies", F.min("anomalies").over(w_monthly))
    .withColumn("sales_norm",
        F.when(F.col("max_sales") == F.col("min_sales"), 1.0)
        .otherwise((F.col("sales") - F.col("min_sales")) / (F.col("max_sales")
- F.col("min_sales"))))
    )
    .withColumn("anomalies_norm",
        F.when(F.col("max_anomalies") == F.col("min_anomalies"), 0.0)
        .otherwise((F.col("anomalies") - F.col("min_anomalies")) /
(F.col("max_anomalies") - F.col("min_anomalies"))))
    )
)

# Compute the final weighted score (e.g., 80% weight for sales, 20% for
anomalies)
weighted_score = normalized_kpis.withColumn(
    "performance_score",
    (0.8 * F.col("sales_norm")) - (0.2 * F.col("anomalies_norm"))
)

(weighted_score.select("order_month", "StoreID", "sales", "anomalies",
"performance_score")
    .write.format("delta").mode("overwrite")
    .saveAsTable(f"{GOLD_SCHEMA}.monthly_store_weighted_score"))

print("\n--- Gold Layer Generation Complete ---")

```

Finally have a look of all the tables that is created.

The screenshot shows the Databricks workspace interface. On the left is a sidebar with navigation options like Workspace, Catalog, Jobs & Pipelines, Compute, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie, Alerts, Query History, SQL Warehouses, Data Engineering, Job Runs, Data Ingestion, AI/ML, Playground, Experiments, Features, Models, and Serving. The main area displays a Python notebook with the following code:

```
full_table_name = f"{{GOLD_SCHEMA}}.{{table_name}}"
print(f"--- Displaying contents of: {{full_table_name}} ---")

# Use display() for a rich, interactive table view in Databricks notebooks
display(spark.table(full_table_name).head(5))
```

Below the code, two table views are shown. The first table is titled "Displaying contents of: sagar_cat_project1.gold.daily_sales_by_region ---" and contains 5 rows of data. The second table is titled "Displaying contents of: sagar_cat_project1.gold.monthly_sales_by_region ---" and contains 4 rows of data. Both tables are refreshed 45 minutes ago.

order_date	Region	1.2 total_sales	total_units_sold	distinct_orders
2023-01-20	East	30315.19999999999...	20	7
2023-01-29	South	32773.42999999999...	28	11
2023-01-02	East	27086.73	19	7
2023-01-11	North	29081.52000000000...	36	11
2023-01-11	East	24138.62999999999...	25	9

order_month	Region	1.2 total_sales	total_units_sold	distinct_orders
2023-01-01T00:00:00.000-00...	East	776331.6000000001	811	252
2023-01-01T00:00:00.000-00...	South	704848.18	738	225
2023-01-01T00:00:00.000-00...	North	955183.64	960	289
2023-01-01T00:00:00.000-00...	West	769714.7799999996	787	234

Microsoft Azure databricks

Workspace

Catalog

For you (All)

- My organization
 - tred_d4
 - system
 - sagar_cat_project1
 - sagar_catalog
 - sagar_catalog2
 - sagar_catalog3
 - praticatalog
 - praticatalog1
 - pratham_02_project
 - pratham_project_1
 - sahil_catalog
 - sahil_catalog_new
 - sanyam_capstone1_catalog
 - sanyam_catalog
 - sanyam_new_catalog
 - Delta Shares Received
 - samples
 - Legacy
 - hive_metastore

File Edit View Run Help Python Tabs ON ☆ Last edit was 1 minute ago

4 2023-01-01T00:00:00.000-00:00 West 769714779999999 787 234

4 rows | 5.16s runtime Refreshed 45 minutes ago

--- Displaying contents of: sagar_cat_project1.gold.daily_anomaly_trend_by_store ---

| order_date | StoreID | anomaly_count | total_transactions | anomaly_rate |
|------------|---------|---------------|--------------------|--------------|
| 2023-01-29 | 18 | 0 | 1 | 0 |
| 2023-01-05 | 15 | 0 | 1 | 0 |
| 2023-01-26 | 10 | 0 | 3 | 0 |
| 2023-01-20 | 16 | 0 | 2 | 0 |
| 2023-01-11 | 17 | 0 | 2 | 0 |

5 rows | 5.16s runtime Refreshed 45 minutes ago

--- Displaying contents of: sagar_cat_project1.gold.daily_conversion_impact_by_store ---

| order_date | StoreID | anomaly_orders | total_orders | successful_orders | success_rate | had_anomaly_day |
|------------|---------|----------------|--------------|-------------------|--------------|-----------------|
| 2023-01-29 | 18 | 0 | 1 | 1 | 1 | 0 |
| 2023-01-05 | 15 | 0 | 1 | 1 | 1 | 0 |
| 2023-01-26 | 10 | 0 | 3 | 3 | 1 | 0 |
| 2023-01-20 | 16 | 0 | 2 | 2 | 1 | 0 |
| 2023-01-11 | 17 | 0 | 2 | 2 | 1 | 0 |

5 rows | 5.16s runtime Refreshed 45 minutes ago

--- Displaying contents of: sagar_cat_project1.gold.monthly_store_performance_tiers ---

Microsoft Azure databricks

Workspace

Catalog

For you (All)

- My organization
 - tred_d4
 - system
 - sagar_cat_project1
 - sagar_catalog
 - sagar_catalog2
 - sagar_catalog3
 - praticatalog
 - praticatalog1
 - pratham_02_project
 - pratham_project_1
 - sahil_catalog
 - sahil_catalog_new
 - sanyam_capstone1_catalog
 - sanyam_catalog
 - sanyam_new_catalog
 - Delta Shares Received
 - samples
 - Legacy
 - hive_metastore

File Edit View Run Help Python Tabs ON ☆ Last edit was 1 minute ago

1 2023-01-29 18 0 1 1 1 0

2 2023-01-05 15 0 1 1 1 0

3 2023-01-26 10 0 3 3 1 0

4 2023-01-20 16 0 2 2 1 0

5 2023-01-11 17 0 2 2 1 0

5 rows | 5.16s runtime Refreshed 45 minutes ago

--- Displaying contents of: sagar_cat_project1.gold.monthly_store_performance_tiers ---

| order_month | StoreID | total_sales | performance_tier |
|-------------------------------|---------|--------------------|------------------|
| 2023-01-01T00:00:00.000-00:00 | 15 | 197270.27999999999 | High |
| 2023-01-01T00:00:00.000-00:00 | 17 | 184736.13000000000 | High |
| 2023-01-01T00:00:00.000-00:00 | 10 | 184143.77 | High |
| 2023-01-01T00:00:00.000-00:00 | 13 | 176925.58999999999 | High |
| 2023-01-01T00:00:00.000-00:00 | 4 | 175538.23 | Medium |

5 rows | 5.16s runtime Refreshed 45 minutes ago

--- Displaying contents of: sagar_cat_project1.gold.monthly_store_weighted_score ---

| order_month | StoreID | total_sales | anomalies | performance_score |
|-------------------------------|---------|--------------------|-----------|--------------------|
| 2023-01-01T00:00:00.000-00:00 | 18 | 173029.89 | 3 | 0.5854040612526225 |
| 2023-01-01T00:00:00.000-00:00 | 11 | 135648.96999999999 | 1 | 0.367662552392394 |
| 2023-01-01T00:00:00.000-00:00 | 5 | 174334.11000000000 | 8 | 0.4835234070595928 |
| 2023-01-01T00:00:00.000-00:00 | 12 | 152687.06 | 2 | 0.4648021774526915 |
| 2023-01-01T00:00:00.000-00:00 | 1 | 163738.09 | 5 | 0.4750479960972901 |

5 rows | 5.16s runtime Refreshed 45 minutes ago

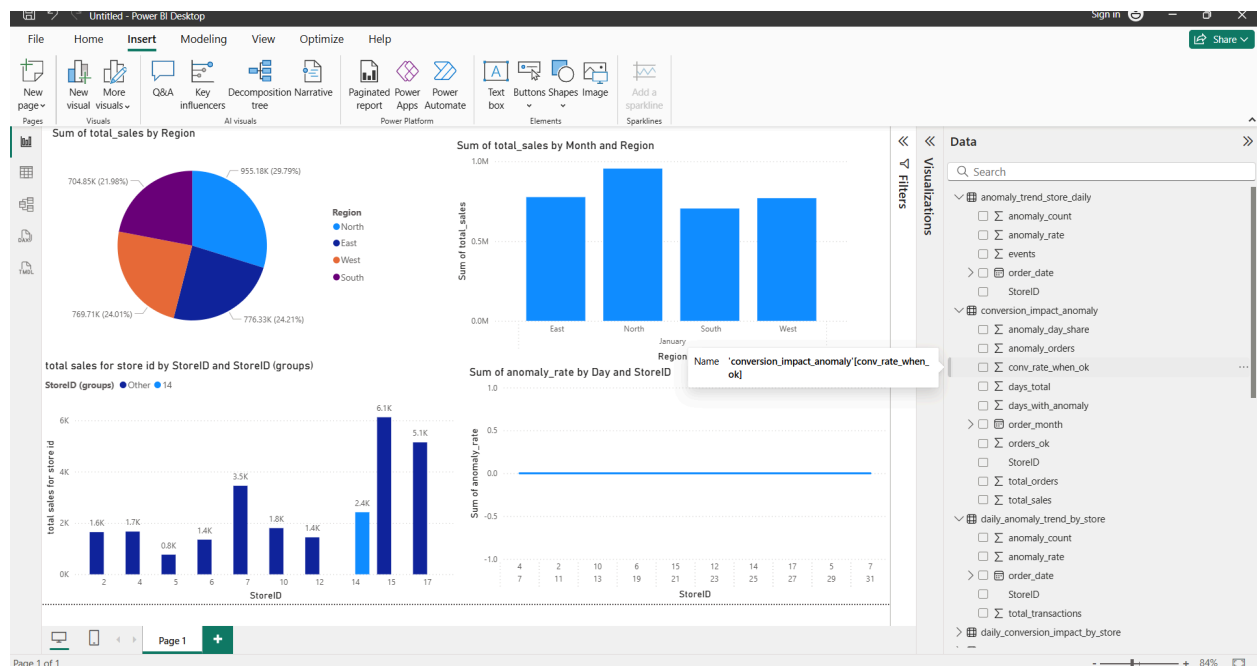
JOBS creation->

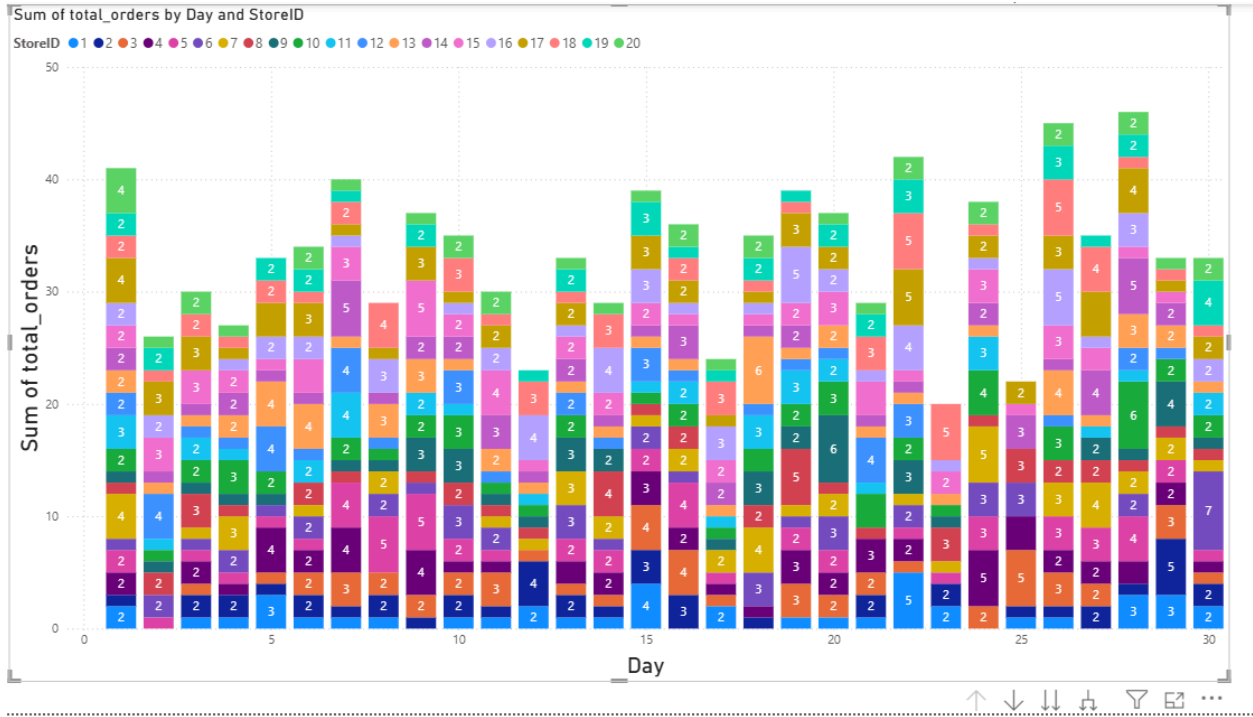
The screenshot displays the Databricks Jobs & Pipelines interface. The main view shows a job run named 'sagar_cap1' with a status of 'Succeeded'. The job is composed of three tasks: 'ingest', 'transform', and 'Gold', all of which have succeeded. The 'ingest' task succeeded 1m 17s ago, 'transform' succeeded 6m 41s ago, and 'Gold' succeeded 53s ago. The job run details on the right show the Job ID as 577889165260315, Job run ID as 162020335769869, and a status of 'Succeeded'. The job was launched manually on August 25, 2025, at 05:31 PM, and ended at 05:40 PM. The duration was 8m 54s, and the execution time was 8m 52s. The job has 0 upstream tables and 1 downstream table. The compute environment is 'kraft200' with a single node of type 'Standard_F4' and release '16.4.6'.

scheduling for regular ingestion and updation of tables->

The screenshot shows the 'Schedules & Triggers' dialog box in the Databricks interface. The 'Trigger Status' is set to 'Active'. The 'Trigger type' is set to 'Scheduled'. The 'Schedule type' is set to 'Simple'. The 'Periodic' schedule is configured with 'Every 1 Day'. The dialog box has 'Cancel' and 'Save' buttons.

Connecting through PowerBi->





Sum of weighted_score by StoreID

